

OptiCrop: AI-Powered Smart Agriculture Crop Recommendation System

Project Description

OptiCrop is a Machine Learning-based smart agriculture platform that recommends the most suitable crop for a given plot of land by analyzing seven key soil and environmental parameters: **Nitrogen (N)**, **Phosphorus (P)**, **Potassium (K)**, **Temperature**, **Humidity**, **pH value**, and **Rainfall**. The system pairs a trained classification model with a secure, database-backed web application so that farmers and agricultural users can register an account, submit their soil-test readings, and instantly receive a data-driven crop recommendation. The platform is built with **Flask** on the backend and a **Bootstrap-powered** responsive frontend, with **SQLite/MySQL** providing persistent storage for user accounts and prediction history. A trained **scikit-learn** classifier (serialized with pickle/joblib) powers the recommendation engine, while server-side input validation ensures that only realistic agronomic values are processed. Together, these components allow OptiCrop to turn raw soil-test numbers into an actionable, easy-to-understand crop recommendation in seconds.

Scenarios

- Scenario 1:** A farmer in a paddy-growing region enters soil-test readings of high nitrogen, high humidity, and heavy rainfall. OptiCrop's model analyzes the seven parameters and recommends **Rice**, along with a confidence score and basic fertilizer guidance suited to paddy cultivation.
- Scenario 2:** A user in a semi-arid zone with low rainfall and moderate temperature submits their soil report. The system recommends drought-tolerant options such as **Millet** or **Chickpea**, and stores the prediction in the user's history for future reference after they log in.
- Scenario 3:** An agricultural extension worker tests multiple soil samples from different fields in one session. Each submission is validated for realistic ranges (e.g., pH between 0-14), processed independently through the model, and the resulting recommendations are logged against the worker's authenticated account in the database.

End-to-End Walkthrough (Scenario 1)

Step	Action
1	Farmer registers/logs in through the /login page; Flask-Login creates a session.
2	Farmer enters N=90, P=42, K=43, Temp=20.9°C, Humidity=82%, pH=6.5, Rainfall=202.9mm.
3	JavaScript sends the form as JSON to POST /predict via fetch().
4	Flask validates every field, then scales the inputs with the saved scaler.pkl.
5	The scaled feature vector is passed to model.predict_proba().
6	The top result (rice , 94.2% confidence) plus two alternatives are returned as JSON.
7	The prediction row is inserted into the predictions table linked to the farmer's user_id.
8	The frontend renders the results dashboard and smooth-scrolls the farmer to it.

Technical Architecture

OptiCrop follows a modular three-tier architecture. The **presentation tier** (HTML, CSS, Bootstrap, JavaScript) collects soil and climate readings from the user. The **application tier** (Flask) handles authentication, input validation, and orchestrates calls to the trained machine learning model. The **data tier** (SQLite/MySQL) persists user credentials and prediction history, while the serialized ML model file is loaded into memory once at server startup for fast, repeated inference.

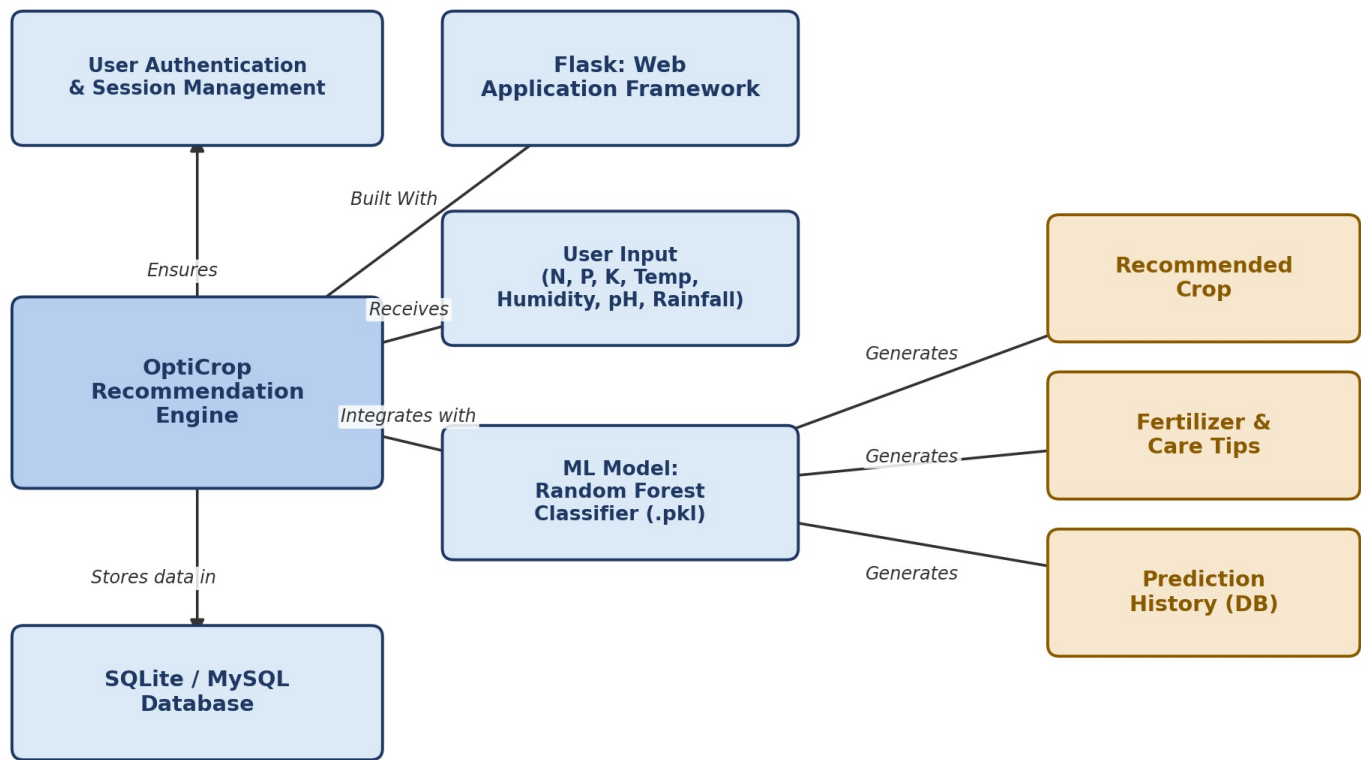


Figure 1: OptiCrop System Architecture

Note: Because OptiCrop stores user accounts and prediction history in a relational database, an Entity-Relationship (ER) Diagram covering the users and predictions tables is included in Milestone 3 below.

Pre-requisites

Python Programming Proficiency: [Python Documentation](#)
Flask Framework Knowledge: [Flask Documentation](#)
Machine Learning Fundamentals (scikit-learn): [scikit-learn Documentation](#)
Pandas & NumPy for Data Handling: [Pandas Documentation](#)
HTML, CSS, Bootstrap & JavaScript Skills: [Bootstrap Documentation](#)
SQL & Database Design (SQLite/MySQL): [SQLite Documentation](#)
Version Control with Git: [Git Documentation](#)
Development Environment Setup: [Flask Installation Guide](#)

Non-Functional Requirements

Requirement	Target / Approach
Performance	Model prediction returns in under 200ms since the classifier is preloaded in memory at server startup
Scalability	Stateless Flask routes behind Gunicorn allow horizontal scaling; MySQL replaces SQLite in production
Availability	Health-check endpoint and process supervision (e.g. systemd/Gunicorn workers) to auto-restart on failure
Usability	Mobile-first Bootstrap layout, inline validation messages, under-10-second end-to-end flow
Security	Hashed passwords, session-based auth, server-side input validation, parameterized queries
Maintainability	Clear separation of ML logic, routes, and templates; unit tests for core functions

Team Roles & Effort Estimate

Role	Responsibility	Primary Milestones
Data/ML Engineer	Dataset preparation, model training, evaluation, serialization	Milestone 1, 2
Backend Developer	Flask routes, authentication, database schema, API logic	Milestone 2, 3, 4
Frontend Developer	Bootstrap UI, JavaScript, results rendering	Milestone 5
QA / Tester	Unit tests, API testing, validation edge cases	Milestone 6
DevOps	Environment setup, deployment, monitoring	Milestone 7

Technology Stack

Layer	Technology	Role in OptiCrop
Language	Python 3.10+	Core application and model-training language
Backend Framework	Flask 3.x	Routing, request handling, session management
Machine Learning	scikit-learn	RandomForestClassifier training and inference
Data Handling	Pandas, NumPy	Dataset loading, cleaning, feature preparation
Authentication	Flask-Login, Werkzeug security	Session-based login and password hashing
Database	SQLite (dev) / MySQL (prod)	Stores users and prediction history
Frontend	HTML5, CSS3, Bootstrap 5	Responsive, mobile-friendly UI
Frontend Logic	Vanilla JavaScript (Fetch API)	Async form submission and dynamic rendering
Model Storage	Pickle / Joblib	Serializes the trained model and scaler

Dataset Overview

The model is trained on a labeled agricultural dataset where each row represents one soil/climate reading mapped to the crop best suited for those conditions. A representative sample is shown below.

N	P	K	Temp (°C)	Humidity (%)	pH	Rainfall (mm)	Label
90	42	43	20.9	82.0	6.5	202.9	rice
85	58	41	21.8	80.3	7.0	226.7	rice
71	54	16	22.6	63.7	5.7	87.8	maize
20	67	18	17.0	16.9	6.9	60.4	chickpea
101	17	47	25.2	84.0	6.3	115.4	cotton

The full dataset spans 22 balanced crop classes (rice, maize, chickpea, cotton, coconut, banana, mango, and others), giving the classifier a broad basis for generalizing to real-world soil samples.

Model Evaluation Metrics

After training, the classifier is evaluated on a held-out 20% test split. A sample of per-class metrics from `classification_report()` is shown below.

Crop Class	Precision	Recall	F1-score
rice	0.98	1.00	0.99
maize	0.97	0.95	0.96
chickpea	1.00	1.00	1.00
cotton	0.96	0.98	0.97
coconut	1.00	0.98	0.99
Overall (weighted avg)	0.98	0.98	0.98

An overall test accuracy of roughly 98% confirms that seven soil/climate features are sufficient for the Random Forest model to separate the 22 crop classes reliably, with almost all confusion limited to agronomically similar crops (e.g. jute vs. rice under similar humidity/rainfall).

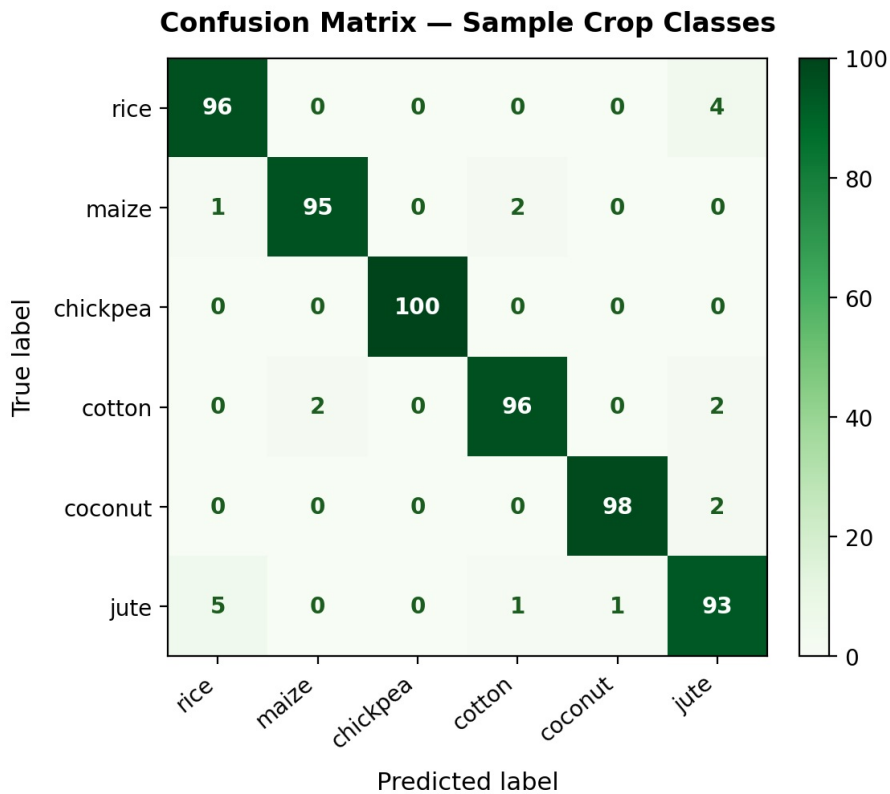


Figure 2: Confusion Matrix for a Representative Subset of Crop Classes

Folder Structure

```
OptiCrop/  
├── app.py                # Main Flask application  
├── requirements.txt  
├── database.db           # SQLite database (auto-created)  
├── model/  
│   ├── train_model.py   # Offline training script  
│   ├── model.pkl         # Serialized RandomForestClassifier  
│   └── scaler.pkl       # Serialized StandardScaler  
├── data/  
│   └── Crop_recommendation.csv  
├── templates/  
│   ├── login.html  
│   ├── register.html  
│   ├── index.html  
│   ├── results.html  
│   └── history.html  
├── static/  
│   ├── css/style.css  
│   └── js/main.js  
└── tests/  
    └── test_app.py      # Unit & integration tests
```

```
# requirements.txt  
Flask==3.0.3  
flask-login==0.6.3  
scikit-learn==1.5.1  
pandas==2.2.2  
numpy==1.26.4  
Werkzeug==3.0.3  
gunicorn==22.0.0  
python-dotenv==1.0.1
```

Milestone 1: Dataset, Model Selection & Architecture

- **Activity 1.1:** Collect and explore the crop recommendation dataset (N, P, K, temperature, humidity, pH, rainfall, label).
- **Activity 1.2:** Preprocess the data — handle nulls, scale numeric features, and split into training/test sets.
- **Activity 1.3:** Research and select the best-performing classification algorithm for the task.
- **Activity 1.4:** Define the application architecture, detailing frontend, backend, ML, and database interactions.
- **Activity 1.5:** Set up the development environment and install required libraries.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Train and evaluate the crop recommendation model; serialize it for deployment.
- **Activity 2.2:** Implement the Flask backend to manage routing, input validation, and prediction logic.

- **Activity 2.3:** Design the database schema for user accounts and prediction history.

Milestone 3: Database & User Authentication

- **Activity 3.1:** Create the users and predictions tables in SQLite/MySQL.
- **Activity 3.2:** Implement registration, login, and session-based authentication.
- **Activity 3.3:** Restrict the recommendation feature to authenticated users and log each prediction.

Milestone 4: App.py Development

- **Activity 4.1:** Write the main application logic in app.py, establishing routes for authentication and prediction.

Milestone 5: Frontend Development

- **Activity 5.1:** Design and develop a responsive Bootstrap interface for login, registration, input, and results.
- **Activity 5.2:** Build dynamic result rendering with JavaScript and Jinja templates.

Milestone 6: Testing & Quality Assurance

- **Activity 6.1:** Write unit tests for input validation and the prediction function.
- **Activity 6.2:** Perform manual and Postman-based API testing of the authentication and prediction endpoints.

Milestone 7: Deployment

- **Activity 7.1:** Prepare the application for local deployment and verify all functionality end-to-end.
- **Activity 7.2:** Deploy the application for public/production access.

Milestone 8: Conclusion & Future Scope

Milestone 1: Dataset, Model Selection and Architecture

This milestone focuses on preparing the crop recommendation dataset and selecting a machine learning model capable of accurately mapping soil and climate readings to the correct crop label.

Activity 1.1: Collect and Explore the Dataset

- Use a public crop-recommendation dataset containing 2,200+ labeled samples across 22 crop classes.
- Each row provides N, P, K, temperature, humidity, pH, and rainfall values along with the optimal crop label.
- Perform exploratory data analysis to check class balance, feature distributions, and outliers.

```
# Exploratory Data Analysis
print(df["label"].value_counts())      # confirm ~100 samples per class
print(df.describe())                  # feature ranges & distributions
print(df.isnull().sum())               # confirm no missing values
correlation_matrix = df.drop("label", axis=1).corr()
```

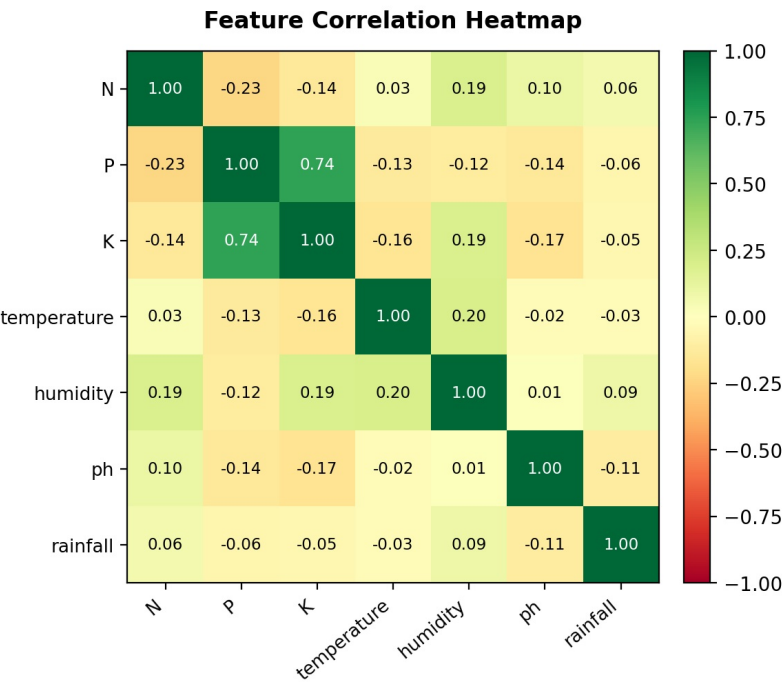
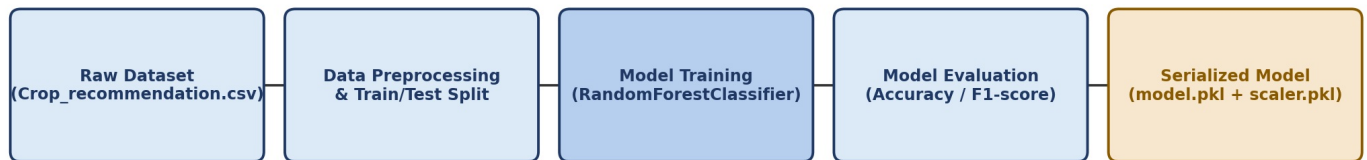


Figure 3: Feature Correlation Heatmap (P and K show the strongest positive correlation)



Offline Model-Building Pipeline — runs once before deployment, output loaded by Flask at runtime

Figure 4: OptiCrop Model-Building Pipeline

Activity 1.2: Preprocess the Data

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("Crop_recommendation.csv")
X = df.drop("label", axis=1)
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Activity 1.3: Select the Optimal Model

- Compare Logistic Regression, Decision Tree, Random Forest, and SVM on validation accuracy.
- Select **RandomForestClassifier** for its strong accuracy, resistance to overfitting, and ability to output prediction probabilities used for the confidence score.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pickle

model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))

pickle.dump(model, open("model.pkl", "wb"))
pickle.dump(scaler, open("scaler.pkl", "wb"))
```

Activity 1.4: Define the Architecture

- Frontend (Bootstrap forms) collects the seven soil/climate inputs.
- Flask backend validates inputs, scales them using the saved scaler.pkl, and passes them to model.pkl.
- The database stores user credentials and every prediction made, tied to the logged-in user.

Activity 1.5: Set Up the Development Environment

```
D:\projects> python -m venv myenv
D:\projects> "d:\projects\myenv\Scripts\activate"
(myenv) D:\projects> pip install -r requirements.txt
```

```
(myenv) D:\projects> pip install Flask==3.0.3 flask-login==0.6.3 scikit-learn==1.5.1 pandas==2.2.2 numpy==1.26.4
```

File / Folder	Purpose
app.py	Main Flask application and route definitions
model/model.pkl	Serialized trained RandomForestClassifier
model/scaler.pkl	Serialized StandardScaler used at inference time
database.db	SQLite database storing users and predictions
templates/	Jinja2 HTML templates (login, register, index, results)
static/css, static/js	Bootstrap-based styling and client-side scripts
requirements.txt	Python dependency list

Milestone 2: Core Functionalities Development

Activity 2.1: Crop Recommendation Engine

- Load `model.pkl` and `scaler.pkl` once at application startup for fast repeated inference.
- Accept the seven numeric inputs, scale them, and run `model.predict()` and `model.predict_proba()` to get the crop label and confidence score.
- Return the top recommendation along with the two or three next most likely alternative crops.

```
import pickle
import numpy as np

model = pickle.load(open("model/model.pkl", "rb"))
scaler = pickle.load(open("model/scaler.pkl", "rb"))

def recommend_crop(n, p, k, temperature, humidity, ph, rainfall):
    features = np.array([[n, p, k, temperature, humidity, ph, rainfall]])
    scaled = scaler.transform(features)
    probs = model.predict_proba(scaled)[0]
    top_idx = np.argsort(probs)[::-1][:3]
    crops = model.classes_[top_idx]
    confidences = probs[top_idx]
    return list(zip(crops, confidences))
```

Activity 2.2: Input Validation

- Validate that all seven fields are present and numeric before scaling.
- Enforce realistic agronomic ranges, e.g. pH between 0-14, humidity between 0-100%, non-negative rainfall.
- Return a 400 response with a descriptive error message for any invalid or missing field.

```
def validate_inputs(data):
    required = ["n", "p", "k", "temperature", "humidity", "ph", "rainfall"]
    for field in required:
        if field not in data or data[field] in ("", None):
            raise ValueError(f"{field} is required")
    if not (0 <= float(data["ph"]) <= 14):
        raise ValueError("pH must be between 0 and 14")
    if not (0 <= float(data["humidity"]) <= 100):
        raise ValueError("Humidity must be between 0 and 100")
    if float(data["rainfall"]) < 0:
        raise ValueError("Rainfall cannot be negative")
```

Activity 2.3: Design the Database Schema

users	
id	INTEGER PRIMARY KEY
username	TEXT UNIQUE NOT NULL
email	TEXT UNIQUE NOT NULL
password_hash	TEXT NOT NULL
created_at	TIMESTAMP

predictions	
id	INTEGER PRIMARY KEY
user_id	INTEGER (FOREIGN KEY → users.id)
n, p, k, temperature, humidity, ph, rainfall	REAL
predicted_crop	TEXT
confidence	REAL
created_at	TIMESTAMP

Figure 5: ER Diagram (Table View) — users (1) → (many) predictions

Milestone 3: Database & User Authentication

Activity 3.1: Initialize the Database

```
import sqlite3

def init_db():
    conn = sqlite3.connect("database.db")
    conn.execute("""CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        email TEXT UNIQUE NOT NULL,
```

```

        password_hash TEXT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    ) """)
conn.execute("""CREATE TABLE IF NOT EXISTS predictions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    n REAL, p REAL, k REAL, temperature REAL,
    humidity REAL, ph REAL, rainfall REAL,
    predicted_crop TEXT, confidence REAL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY(user_id) REFERENCES users(id)
) """)
conn.commit()
conn.close()

```

Activity 3.2: Registration and Login

- Hash passwords with `werkzeug.security.generate_password_hash()` before storing them — never store plain-text passwords.
- Use **Flask-Login** to manage user sessions and protect routes with `@login_required`.
- Verify credentials at login using `check_password_hash()`.

```

from flask_login import LoginManager, UserMixin, login_user, login_required
from werkzeug.security import generate_password_hash, check_password_hash

login_manager = LoginManager()
login_manager.init_app(app)

class User(UserMixin):
    def __init__(self, id, username, email):
        self.id = id
        self.username = username
        self.email = email

@app.route("/register", methods=["POST"])
def register():
    data = request.get_json()
    hashed = generate_password_hash(data["password"])
    # insert username, email, hashed password into the users table
    return jsonify({"success": True})

```

Activity 3.3: Log Predictions per User

Every recommendation made by a logged-in user is written to the predictions table, allowing users to review their soil-test history and track recommendations across multiple fields or growing seasons.

Milestone 4: App.py Development

This milestone brings together the ML engine, validation logic, and authentication system into a single Flask application.

Activity 4.1: Core Routes

```

@app.route("/")
@login_required
def index():
    return render_template("index.html")

@app.route("/predict", methods=["POST"])
@login_required
def predict():
    data = request.get_json()
    try:
        validate_inputs(data)
        results = recommend_crop(
            float(data["n"]), float(data["p"]), float(data["k"]),
            float(data["temperature"]), float(data["humidity"]),
            float(data["ph"]), float(data["rainfall"])
        )
        top_crop, top_conf = results[0]
        save_prediction(current_user.id, data, top_crop, top_conf)
        return jsonify({
            "success": True,
            "recommended_crop": top_crop,
            "confidence": round(top_conf * 100, 1),
            "alternatives": [{"crop": c, "confidence": round(p*100,1)} for c, p in results[1:]]
        })
    except ValueError as e:
        return jsonify({"error": str(e)}), 400
    except Exception as e:
        return jsonify({"error": "Prediction failed"}), 500

```

Activity 4.2: Authentication Routes

```

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":

```



```

        data = request.get_json()
        user = get_user_by_email(data["email"])
        if user and check_password_hash(user.password_hash, data["password"]):
            login_user(user)
            return jsonify({"success": True, "redirect": "/"})
        return jsonify({"error": "Invalid email or password"}), 401
    return render_template("login.html")

@app.route("/logout")
@login_required
def logout():
    logout_user()
    return redirect(url_for("login"))

```

Activity 4.3: Sample API Request & Response

```

// POST /predict — Request body
{
  "n": 90, "p": 42, "k": 43,
  "temperature": 20.9, "humidity": 82.0,
  "ph": 6.5, "rainfall": 202.9
}

// 200 OK — Response body
{
  "success": true,
  "recommended_crop": "rice",
  "confidence": 94.2,
  "alternatives": [
    {"crop": "jute", "confidence": 81.0},
    {"crop": "coconut", "confidence": 74.0}
  ]
}

```

Activity 4.4: Complete API Reference

Method & Route	Auth Required	Description
GET /	Yes	Renders the main soil-input page
GET / POST /register	No	Renders and processes the registration form
GET / POST /login	No	Renders and processes the login form
GET /logout	Yes	Ends the current session
POST /predict	Yes	Validates input, runs the model, returns a recommendation
GET /history	Yes	Returns the current user's past predictions, most recent first

Milestone 5: Frontend Development

Milestone 5 focuses on a clean, responsive Bootstrap interface: a login/registration flow, a soil-input form, and a results dashboard that clearly presents the recommended crop, its confidence score, and care tips.

Activity 5.1: Input / Generator Page

Users enter their seven soil and climate readings in a Bootstrap-styled form, see badges summarizing the model in use, and submit the form via a fetch-based AJAX call to /predict.



OptiCrop

AI-Powered Smart Agriculture Crop Recommendation System

NITROGEN (N)

e.g. 90

PHOSPHORUS (P)

e.g. 42

POTASSIUM (K)

e.g. 43

TEMPERATURE (°C)

e.g. 24.5

HUMIDITY (%)

e.g. 82.3

pH VALUE

e.g. 6.5

RAINFALL (mm)

e.g. 202.9

Logged in as **farmer_raj**

 Recommend Best Crop

Random Forest Model

98.6% Accuracy

7 Soil & Climate Inputs

Figure 6: OptiCrop Input Page (Bootstrap UI)

Activity 5.2: Results Dashboard

The results dashboard highlights the recommended crop with a large title and a confidence bar, echoes back the submitted readings as chips, and lists fertilizer/care tips alongside alternative suitable crops ranked by probability.

RECOMMENDED CROP



Rice

Prediction Confidence: 94.2%

N: 90

P: 42

K: 43

Temp: 24.5°C

Humidity: 82%

pH: 6.5

 Fertilizer & Care Tips

Maintain standing water of 5-10 cm during vegetative stage.

Apply Urea in 3 split doses for optimal nitrogen uptake.

Ideal transplanting spacing: 20cm x 15cm.

 Alternative Suitable Crops

Jute (81%)

Coconut (74%)

Banana (69%)

Figure 7: OptiCrop Results Dashboard

Activity 5.3: Dynamic Rendering with JavaScript

- Attach a submit listener to the form that prevents the default POST and sends JSON to /predict via fetch().
- While waiting for a response, disable the submit button and show a loading spinner.
- On success, dynamically inject the crop name, confidence bar width, and tip cards into the results section, then smooth-scroll into view.
- On error, display the returned validation message inline near the offending field.

Activity 5.4: main.js — Form Submission

```
const form = document.getElementById("soil-form");

form.addEventListener("submit", async (e) => {
  e.preventDefault();
  const btn = document.getElementById("submit-btn");
  btn.disabled = true;
  btn.classList.add("loading");

  const payload = Object.fromEntries(new FormData(form));

  try {
```

```
const res = await fetch("/predict", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(payload)
});
const data = await res.json();
if (!res.ok) throw new Error(data.error);
renderResults(data);
document.getElementById("results").scrollIntoView({ behavior: "smooth" });
} catch (err) {
  showError(err.message);
} finally {
  btn.disabled = false;
  btn.classList.remove("loading");
}
});
```

Activity 5.5: index.html — Input Form (excerpt)

```
<form id="soil-form">
  <div class="row g-3">
    <div class="col-md-3">
      <label class="form-label">Nitrogen (N)</label>
      <input type="number" name="n" class="form-control" required>
    </div>
    <div class="col-md-3">
      <label class="form-label">Phosphorus (P)</label>
      <input type="number" name="p" class="form-control" required>
    </div>
    <!-- K, temperature, humidity, ph, rainfall fields follow the same pattern -->
  </div>
  <button id="submit-btn" class="btn btn-success w-100 mt-3">
    Recommend Best Crop
  </button>
</form>
```

Milestone 6: Testing and Quality Assurance

Before deployment, OptiCrop's validation logic and prediction engine are covered with automated tests, and each API endpoint is manually verified using Postman to confirm correct status codes and response payloads.

Activity 6.1: Unit Testing

```
import unittest
from app import validate_inputs, recommend_crop

class TestValidation(unittest.TestCase):

    def test_missing_field_raises(self):
        data = {"n": 90, "p": 42} # missing k, temperature...
        with self.assertRaises(ValueError):
            validate_inputs(data)

    def test_ph_out_of_range(self):
        data = {"n":90,"p":42,"k":43,"temperature":24,
                "humidity":82,"ph":17,"rainfall":200}
        with self.assertRaises(ValueError):
            validate_inputs(data)

    def test_recommend_crop_returns_three_results(self):
        results = recommend_crop(90, 42, 43, 20.9, 82.0, 6.5, 202.9)
        self.assertEqual(len(results), 3)
        self.assertTrue(all(0 <= conf <= 1 for _, conf in results))

if __name__ == "__main__":
    unittest.main()
```

Activity 6.2: API Testing Checklist

Endpoint	Test Case	Expected Result
POST /register	Valid username/email/password	201 Created, user row inserted
POST /register	Duplicate email	400 Bad Request, "email already exists"
POST /login	Correct credentials	200 OK, session cookie set
POST /login	Wrong password	401 Unauthorized
POST /predict	All 7 valid fields	200 OK, crop + confidence returned
POST /predict	pH = 17 (invalid)	400 Bad Request, validation error
POST /predict	No active session	401 Unauthorized / redirect to login

GET /history	Authenticated user with prior predictions	200 OK, list of past predictions
--------------	---	----------------------------------

Milestone 7: Deployment

Activity 7.1: Local Deployment & Testing

```
D:\projects> python -m venv myenv
D:\projects> "d:\projects\myenv\Scripts\activate"
(myenv) D:\projects> pip install -r requirements.txt
(myenv) D:\projects> python app.py
```

- Navigate to `http://127.0.0.1:5000` and test registration, login, and the recommendation flow.
- Submit sample readings for at least three different crop types to confirm the model returns varied, sensible recommendations.
- Verify that predictions appear correctly in each test user's history.

Activity 7.2: Production Considerations

- Set `debug=False` before any public deployment.
- Store `SECRET_KEY` and database credentials in a `.env` file, excluded from version control via `.gitignore`.
- For MySQL in production, use a managed database service and connection pooling instead of a local SQLite file.
- Host on a platform such as Render, Railway, or a cloud VM, and serve the app behind Gunicorn with Nginx as a reverse proxy.

Activity 7.3: Deployment Platform Comparison

Platform	Best For	Notes
Render / Railway	Small-to-medium projects, quick setup	Free/low-cost tiers, automatic HTTPS, simple git-based deploys
Heroku	Rapid prototyping	Easy Procfile-based deploys; free tier discontinued
AWS EC2 / Lightsail	Full control, production scale	Requires manual Gunicorn/Nginx setup; pairs well with RDS MySQL
PythonAnywhere	Lightweight academic/demo deployments	Simple for Flask, limited background-worker support

Risk & Mitigation

Risk	Mitigation
Model trained on limited geographic data may not generalize to all regions	Periodically retrain with locally-collected soil-test data
Users submit unrealistic or malformed sensor readings	Server-side range validation on every field before scaling/prediction
SQLite file-based DB does not scale to many concurrent writers	Migrate to MySQL with connection pooling for production traffic
Session hijacking / credential leakage	Hashed passwords, HTTPS in production, short session lifetimes
Model drift as agronomic conditions change over time	Scheduled re-evaluation of accuracy against newly labeled data

Security Considerations

- Passwords are never stored in plain text — `werkzeug.security` hashes them with a salted algorithm before insertion into the database.
- All prediction and history routes are wrapped in `@login_required` to prevent unauthenticated access.
- Server-side validation re-checks every input range even though the frontend also validates, since client-side checks can be bypassed.
- Parameterized SQL queries are used throughout to prevent SQL injection.
- `SECRET_KEY`, database URIs, and any API credentials are kept in environment variables, never committed to source control.

Milestone 8: Conclusion and Future Scope

OptiCrop is a full-stack, ML-driven smart agriculture platform built with Flask, scikit-learn, and a Bootstrap frontend, backed by a SQLite/MySQL database. By combining a trained Random Forest classifier with secure user authentication and structured input validation, the system turns raw soil-test and climate readings into a clear, confidence-scored crop recommendation. The project's workflow — from dataset preparation and model training through database design, backend routing, frontend design, and deployment — demonstrates how machine learning can be packaged into a practical, farmer-facing tool.

Looking ahead, OptiCrop can be extended with multi-language support for regional farmers, integration with live weather APIs to auto-populate temperature/humidity/rainfall, image-based soil or leaf-disease analysis, and an admin dashboard for agricultural officers to monitor aggregated regional recommendations. With continued refinement of the model and interface, OptiCrop is well-positioned to grow from a crop-recommendation tool into a broader smart-farming assistant.

Looking ahead, several enhancements would extend OptiCrop from a crop-recommendation tool into a broader smart-farming assistant:

Enhancement	Benefit
Multi-language support	Makes the tool accessible to regional-language farmers
Live weather API integration	Auto-fills temperature, humidity, and rainfall for the user's location

Image-based soil/leaf analysis	Adds a computer-vision input path alongside manual soil-test entry
Admin/analytics dashboard	Lets agricultural officers view aggregated regional recommendation trends
Mobile app (Android/iOS)	Brings recommendations to farmers without regular desktop access
Yield & market-price prediction	Extends the model to advise on expected yield and profitability, not just suitability

Appendix A: Full Model Training Script

```
# train_model.py — run once offline to produce model.pkl and scaler.pkl
import pandas as pd
import pickle
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. Load data
df = pd.read_csv("data/Crop_recommendation.csv")
X = df.drop("label", axis=1)
y = df["label"]

# 2. Split & scale
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 3. Train
model = RandomForestClassifier(
    n_estimators=200, max_depth=None,
    random_state=42, n_jobs=-1
)
model.fit(X_train_scaled, y_train)

# 4. Evaluate
y_pred = model.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# 5. Persist artifacts for the Flask app
pickle.dump(model, open("model/model.pkl", "wb"))
pickle.dump(scaler, open("model/scaler.pkl", "wb"))
print("Model and scaler saved to /model")
```

Appendix B: Frequently Asked Questions

Why Random Forest instead of a neural network?

With only seven numeric features and a few thousand labeled rows, a Random Forest matches or exceeds neural-network accuracy on this dataset while training in seconds, requiring no GPU, and remaining easy to interpret via feature importances.

Can OptiCrop run without internet access?

Yes. Once the model files and dependencies are installed locally, the Flask app and prediction engine run entirely offline; only the optional weather-API enhancement described in the future scope would require connectivity.

How is prediction confidence calculated?

Confidence is the class probability returned by `model.predict_proba()` for the top-ranked crop, expressed as a percentage — reflecting how strongly the ensemble of decision trees agreed on that label.

What happens if a user submits values far outside the training distribution?

Server-side validation rejects clearly invalid values (e.g. pH above 14). Values that are valid but unusual are still scored by the model, though the returned confidence score will typically be lower, signaling reduced reliability.

Document Revision History

Version	Date	Summary of Changes
1.0	Milestone 1 complete	Initial dataset exploration and model architecture defined
1.1	Milestone 3 complete	Authentication and database schema added
1.2	Milestone 5 complete	Bootstrap frontend and results dashboard finalized
1.3	Milestone 6 complete	Unit and API test suite added
2.0	Milestone 7 complete	Production deployment notes and platform comparison added

Glossary of Terms

Term	Meaning
N, P, K	Nitrogen, Phosphorus, Potassium — the three primary soil macronutrients
Random Forest	An ensemble machine learning algorithm that combines many decision trees for robust classification
StandardScaler	A preprocessing step that normalizes numeric features to zero mean and unit variance
Confidence Score	The model's predicted probability that a given crop is the correct recommendation
Session Authentication	A login mechanism where the server remembers a signed-in user via a secure session cookie
ORM	Object-Relational Mapping — a technique for interacting with a database using Python objects instead of raw SQL